

1. Data Collection:

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor, IsolationForest
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import silhouette_score
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, classification_report

# load the dataset in dataframe
df = pd.read_csv(r"C:\Users\Public\Downloads\laptop_prices.csv", encoding='utf-8-sig')
df.head()
```

2. Data Cleaning & Pre-Processing:

```
# Convert month column to datetime format
df['Ram'] = df['Ram'].astype(str).str.replace('GB', "").astype(int)
df['Weight'] = df['Weight'].astype(str).str.replace('kg', "").astype(float)
df['PPI'] = ((df['ScreenW']**2 + df['ScreenH']**2)**0.5) / df['Inches']
df['TotalStorage'] = df['PrimaryStorage'] + df['SecondaryStorage']
df['Touchscreen'] = df['Touchscreen'].apply(lambda x: 1 if x == 'Yes' else 0)
df['IPSPanel'] = df['IPSPanel'].apply(lambda x: 1 if x == 'Yes' else 0)
df['RetinaDisplay'] = df['RetinaDisplay'].apply(lambda x: 1 if x == 'Yes' else 0)
df['Screen'] = df['Screen'].apply(lambda x: 1 if x == 'Full HD' else (2 if x == '4K Ultra HD' else 0))

cat_cols = [col for col in ['Company', 'TypeName', 'OS', 'CPU_company', 'PrimaryStorageType', 'SecondaryStorageType', 'GPU_company'] if col in df.columns]

df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
df.to_csv(r"C:\Users\Public\Downloads\laptop_prices.csv", index=False)
```

Check for missing values

```
df.isnull().sum()
```

3. Model Development:

Actual vs Predicted Laptop Prices

```
features = ['Touchscreen', 'RetinaDisplay', 'CPU_freq', 'Ram', 'Weight', 'TotalStorage', 'PPI',
'Screen'] + \

    [col for col in df.columns if col.startswith(('Company_', 'CPU_company_', 'GPU_company_',
'TypeName_', 'OS_',

                                                'PrimaryStorageType_', 'SecondaryStorageType_'))]

X = df[features]
y = df['Price_euros']
X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(X, y, test_size=0.2, random_state=42)
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train_rf, y_train_rf)
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test_rf, y=rf_pred, alpha=0.6, color='green')
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Laptop Prices")
plt.grid(True)
plt.plot([min(y_test_rf), max(y_test_rf)], [min(y_test_rf), max(y_test_rf)], color='red', linestyle='--',
label='Perfect Prediction')
plt.legend()
plt.show()
```

Top 10 Important Features

```
importances = rf.feature_importances_
features = X.columns
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(8, 5))
sns.barplot(x=importances[indices][:10], y=features[indices][:10], palette='viridis')
plt.title("Top 10 Important Features")
plt.xlabel("Feature Importance Score")
```

```

plt.ylabel("Features")
plt.show()

# RAM vs Price
means_features = ['Ram', 'Weight', 'Price_euros']
X_kmeans = df[kmeans_features]
scaler_kmeans = StandardScaler()
X_scaled_kmeans = scaler_kmeans.fit_transform(X_kmeans)
kmeans = KMeans(n_clusters=2, random_state=42)
df['Cluster'] = clusters
df['Category'] = df['Cluster'].map({0: 'Budget', 1: 'Premium'})
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='Ram', y='Price_euros', hue='Cluster', palette='Set1')
plt.title("RAM vs Price")
plt.xlabel("RAM")
plt.ylabel("Price")
plt.grid(True)
plt.legend(title='Cluster', loc='upper right', bbox_to_anchor=(1.15, 1))
plt.tight_layout()
plt.show()

# Weight vs Price
iso_features = ['Weight', 'Inches', 'Price_euros', 'TotalStorage']
X_iso = df[iso_features]
scaler_iso = StandardScaler()
X_scaled_iso = scaler_iso.fit_transform(X_iso)
iso = IsolationForest(n_estimators=100, contamination=0.01, random_state=42)
outliers = iso.fit_predict(X_scaled_iso)
df['Outlier'] = outliers
plt.figure(figsize=(8, 5))
sns.scatterplot(x='Weight', y='Price_euros', hue='Outlier', data=df, palette='Set1', alpha=0.8)
plt.title("Weight vs Price")
plt.xlabel("Weight")

```

```

plt.ylabel("Price")
plt.grid(True)
plt.tight_layout()
plt.legend(title='Outlier', loc='upper right', bbox_to_anchor=(1.15, 1))
plt.show()

# Laptop Price Category
labels = ['Budget', 'Midrange', 'Premium']
df['PriceCategory'] = pd.cut(df['Price_euros'], bins=bins, labels=labels)
dt_features = ['Ram', 'Weight', 'CPU_freq', 'PrimaryStorage', 'SecondaryStorage']
X_dt = df[dt_features]
y_dt = df['PriceCategory']
X_train_dt, X_test_dt, y_train_dt, y_test_dt = train_test_split(X_dt, y_dt, test_size=0.2,
random_state=42)
scaler_dt = StandardScaler()
X_train_dt_scaled = scaler_dt.fit_transform(X_train_dt)
X_test_dt_scaled = scaler_dt.transform(X_test_dt)
model_dt = DecisionTreeClassifier(random_state=42, class_weight='balanced')
model_dt.fit(X_train_dt_scaled, y_train_dt)
print("\nClassification Report:\n", classification_report(y_test_dt, y_pred_dt))
report = classification_report(y_test_dt, y_pred_dt, output_dict=True)
df_report = pd.DataFrame(report).transpose().loc[['Budget', 'Midrange', 'Premium']]
df_melted = df_report[['precision', 'recall', 'f1-score']].reset_index().melt(id_vars='index')
df_melted.columns = ['Category', 'Metric', 'Score']
plt.figure(figsize=(7, 5))
sns.barplot(data=df_melted, x='Category', y='Score', hue='Metric')
plt.title("Laptop Price Category")
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0)
plt.show()

```

4. Evaluation:

Random Forest Evaluation

```
print("\n Random Forest Evaluation:")  
print("MSE:", mean_squared_error(y_test_rf, rf_pred)) # Mean Squared Error  
print("R2 Score:", r2_score(y_test_rf, rf_pred))
```

Silhouette Score for KMeans

```
clusters = kmeans.fit_predict(X_scaled_kmeans)  
silhouette = silhouette_score(X_scaled_kmeans, clusters)  
print("\nSilhouette Score for KMeans with k=2:", silhouette)
```

Decision Tree Evaluation

```
pred_dt = model_dt.predict(X_test_dt_scaled)  
print("\nDecision Tree Classifier Evaluation:")  
print("Accuracy:", accuracy_score(y_test_dt, y_pred_dt)) # Accuracy  
print("Classification Report:\n", classification_report(y_test_dt, y_pred_dt, target_names=['Budget',  
'Midrange', 'Premium']))
```